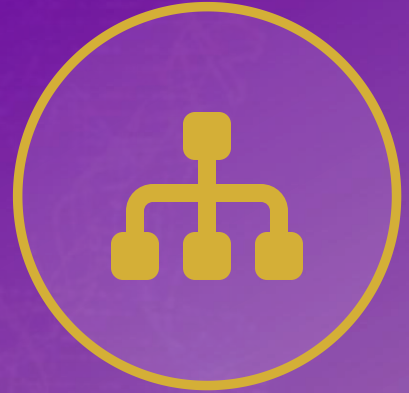
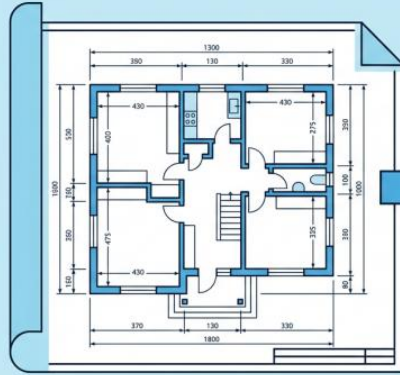


All cards in one array



UML Diagram



Source Code



ClassName

- id: int
- name: string
- # email: string
- + getName(): string
- # print(): void
- calculate(): double

- + public
- private
- # protected

```
class Person {  
protected:  
    int id;  
    string name;  
  
public:  
    Person(int id, const string& name);  
    ~Person() = default; // Destructor (non-v  
  
    int getId() const { return id; }  
    string getName() const { return name; }  
    void print() const;  
};
```

Types of Relationships

01

Inheritance: "Is-A" / 继承: "是一个"

A student is a person (inherits person attributes)

class Student : public Person

02

Composition: "Has-A" (Strong) / 组合: "有一个" (强)

A human has a heart

The heart cannot exist independently. If the human dies, the heart dies too

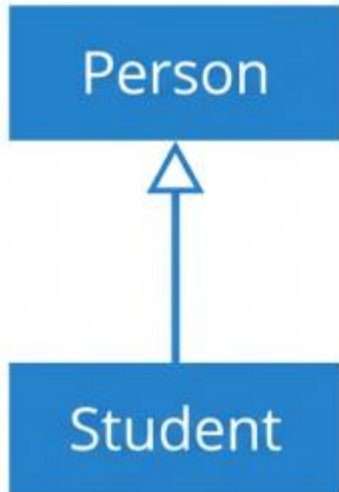
03

Aggregation: "Has-A" (Weak) / 聚合: "有一个" (弱)

A university has students.

But students exist independently If the university closes, students still exist

Inheritance



is-a

Composition



has-a (strong) - dies together

Aggregation



has-a (weak) - independent

Public vs. Private Inheritance

Base Access	Public Inherit.	Private Inherit.
public	public	private
protected	protected	private

Public Inheritance

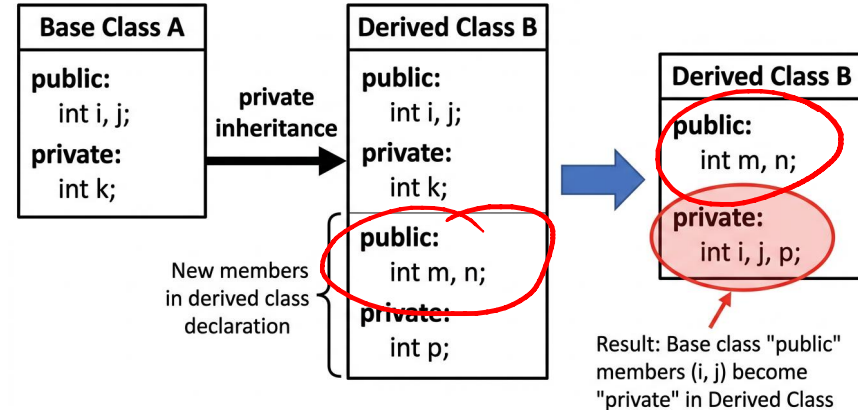
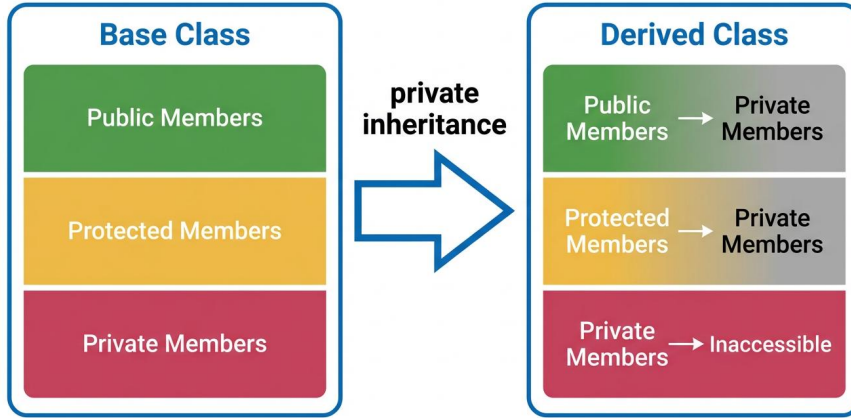
- Models "is-a" relationship
- Supports polymorphism
- Preserves base interface

Private Inheritance

- "Implemented-in-terms-of"
- Implementation reuse only
- Hides base interface

Guideline: Prefer Public for domain modeling; Private for hidden implementation details.

Private Inheritance



Protected Inheritance

```
class A {  
protected:  
    int x;  
};  
  
int main() {  
    A a;  
    a.x = 5;    //Is it correct?  
}
```

```
class A {  
protected:  
    int x;  
};  
class B: public A{  
public:  
    void function();  
};  
void B::function() {  
    x = 5;    //Correct  
}
```

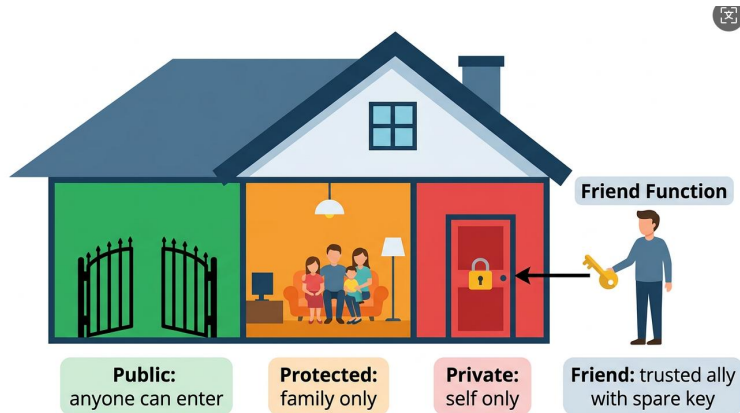
Friend

Imagine you have:

- **Private bedroom** (private members): Only you can enter
- **Protected living room** (protected members): Family can enter
- **Public yard** (public members): Anyone can enter

Now imagine your **best friend** or **trusted neighbor**:

- They're not family (not inheritance)
- But you give them a **spare key** to your home
- They can enter your **private bedroom** when needed



Friend Function: External Function with Private Access

```
class BankAccount {
private:
    double balance;    // Private - only the account can modify
    string password;   // Private - sensitive data

public:
    BankAccount(double initial) : balance(initial) {}

    // Regular public interface
    double getBalance() const { return balance; }

    // Declare external function as FRIEND
    // 声明外部函数为友元
    friend bool transferFunds(BankAccount& from, BankAccount& to, double amount);
    friend class BankManager; // Entire class is a friend!
};

// External function - but can access private members!
// 外部函数 - 但可以访问私有成员!
bool transferFunds(BankAccount& from, BankAccount& to, double amount) {
    // This function is NOT a member of BankAccount
    // 这个函数不是BankAccount的成员
    // But it can access PRIVATE members!
    // 但它可以访问私有成员!

    if (from.balance >= amount) { // ✓ Accessing private 'balance'!
        from.balance -= amount;
        to.balance += amount;    // ✓ Accessing private 'balance'!
        return true;
    }
    return false;
}
```

Friend Class: Entire Class Gets Access

```
class BankManager; // Forward declaration

class BankAccount {
private:
    double balance;
    int accountNumber;

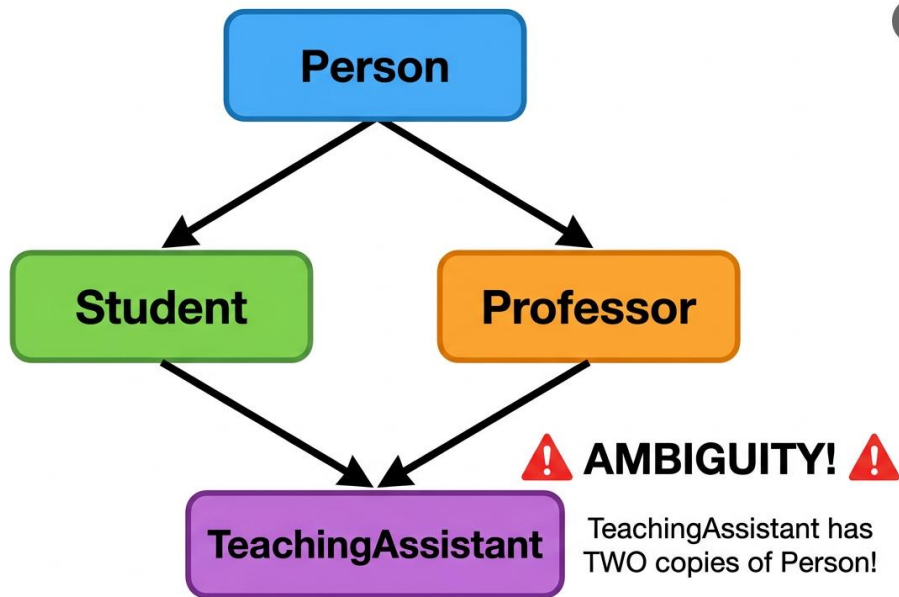
public:
    BankAccount(int num, double initial)
        : accountNumber(num), balance(initial) {}

    // Grant ALL members of BankManager access to ALL private members
    // 授予BankManager所有成员访问所有私有成员的权限
    friend class BankManager;
};

class BankManager {
public:
    // Can access private data of BankAccount directly!
    // 可以直接访问BankAccount的私有数据!
    void auditAccount(const BankAccount& account) {
        cout << "Account #" << account.accountNumber << endl; // ✓ Private!
        cout << "Balance: $" << account.balance << endl;      // ✓ Private!
    }

    void freezeAccount(BankAccount& account) {
        // Direct modification of private data!
        account.balance = 0; // ✓ Can modify private!
        cout << "Account frozen!" << endl;
    }
};
```

The Diamond Problem / 菱形问题



```
class Person {
public:
    string name;
    Person(const string& n) : name(n) {}
};

class Student : public Person { // ← Inherits Person
public:
    double gpa;
    Student(const string& n, double g) : Person(n), gpa(g) {}
};

class Professor : public Person { // ← Also inherits Person!
public:
    string department;
    Professor(const string& n, const string& d) : Person(n), department(d) {}
};

// Teaching Assistant is both Student AND Professor
class TA : public Student, public Professor {
public:
    TA(const string& n, double g, const string& d)
        : Student(n, g), Professor(n, d) // ⚠️ PROBLEM: Person constructed TWICE!
    {}
};

// What happens here?
TA ta("Alice", 3.8, "CS");
// ta.name = ??? // ERROR: ambiguous! Which name? Student's or Professor's?
```

Virtual Base Class

```
class Person {
public:
    string name;
    int id;
    Person(const string& n, int i) : name(n), id(i) {}
};

// Use VIRTUAL inheritance
class Student : virtual public Person { // ← virtual!
public:
    double gpa;
    Student(const string& n, int i, double g)
        : Person(n, i), gpa(g) {}
};

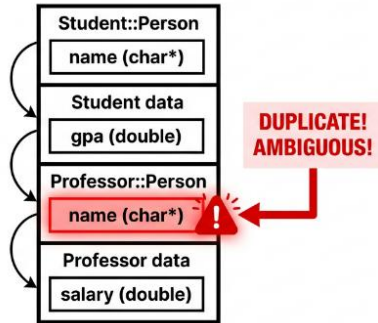
class Professor : virtual public Person { // ← virtual!
public:
    string department;
    Professor(const string& n, int i, const string& d)
        : Person(n, i), department(d) {}
};
```

```
// Now TA has only ONE Person part
class TA : public Student, public Professor {
public:
    // MUST initialize virtual base directly!
    TA(const string& n, int i, double g, const string& d)
        : Person(n, i)           // ← Initialize virtual base FIRST
        , Student(n, i, g)       // Virtual base init ignored here
        , Professor(n, i, d)     // Virtual base init ignored here
    {}
};

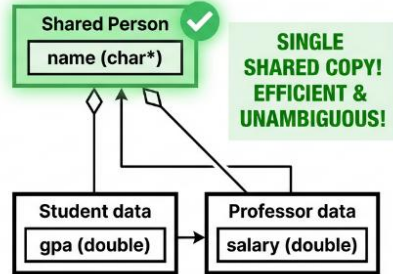
// Usage:
TA ta("Alice", 1001, 3.8, "CS");
cout << ta.name;           // ✓ OK: Unambiguous! Single copy
cout << ta.id;             // ✓ OK: Only one id
```

DIAGRAM: MEMORY LAYOUT COMPARISON – MULTIPLE INHERITANCE

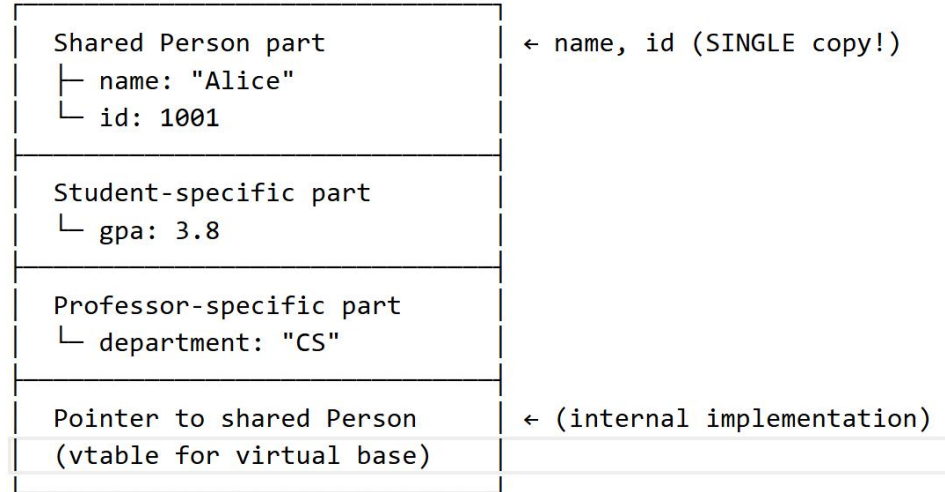
WITHOUT virtual (BROKEN)



WITH virtual (FIXED)



TA Object in Memory:



```

#include <iostream>
using namespace std;
class Base1 { //基类Base1定义
public:
    void display() const {
        cout << "Base1::display()" << endl;
    }
};
class Base2: public Base1 { //公有派生类Base2定义
public:
    void display() const {
        cout << "Base2::display()" << endl;
    }
};
class Derived: public Base2 { //公有派生类Derived定义
public:
    void display() const {
        cout << "Derived::display()" << endl;
    }
};

```

```

void fun(Base1 *ptr) {           //参数为指向基类对象的指针
    ptr->display();              // "对象指针->成员名"
}
int main() {                    //主函数
    Base1 base1;                //声明Base1类对象
    Base2 base2;                //声明Base2类对象
    Derived derived;            //声明Derived类对象

    fun(&base1);                 //用Base1对象的指针调用fun函数
    fun(&base2);                 //用Base2对象的指针调用fun函数
    fun(&derived);               //用Derived对象的指针调用fun函数

    return 0;
}

```

运行结果：

```

Base1::display()
Base1::display()
Base1::display()

```

```

1 class B{
2 public:
3     void mf() {
4         cout<<"B"<<endl;
5     }
6 };
7
8 class D:public B{
9 public:
10    void mf() //名字覆盖
11        cout<<"D"<<endl;
12    }
13 };

```

```

int main() {
    D x;
    B *pb = &x;
    cout << "pb: " << pb << endl;
    pb->mf();

    D *pd = &x;
    cout << "pd: " << pb << endl;
    pd->mf();

    return 0;
}

```

```

"C:\work\C++
pb: 0x6dfef7
B
pd: 0x6dfef7
D
Process returned
Press any key

```

```
#include <iostream>
using namespace std;

class Base1 {
public:
    virtual void display() const; //虚函数
};

void Base1::display() const {
    cout << "Base1::display()" << endl;
}

class Base2::public Base1 {
public:
    virtual void display() const;
};

void Base2::display() const {
    cout << "Base2::display()" << endl;
}

class Derived: public Base2 {
public:
    virtual void display() const;
};

void Derived::display() const {
    cout << "Derived::display()" << endl;
}
```

```
void fun(Base1 *ptr) {
    ptr->display();
}
```

```
int main() {
    Base1 base1;
    Base2 base2;
    Derived derived;
    fun(&base1);
    fun(&base2);
    fun(&derived);
    return 0;
}
```

运行结果:

```
Base1::display()
Base2::display()
Derived::display()
```


Abstract Class

```
//8_6.cpp
#include <iostream>
using namespace std;

class Base1 {
public:
    virtual void display() const = 0;           //Abstract class
};

class Base2: public Base1 {
public:
    virtual void display() const;
};

void Base2::display() const {
    cout << "Base2::display()" << endl;
}
```



Abstract class

```
class Derived: public Base2 {
public:
    virtual void display() const; //覆盖基类的虚函数
};
void Derived::display() const {
    cout << "Derived::display()" << endl;
}
void fun(Base1 *ptr) {
    ptr->display();
}
int main() {
    Base2 base2;
    Derived derived;
    fun(&base2);
    fun(&derived);
    return 0;
}
```



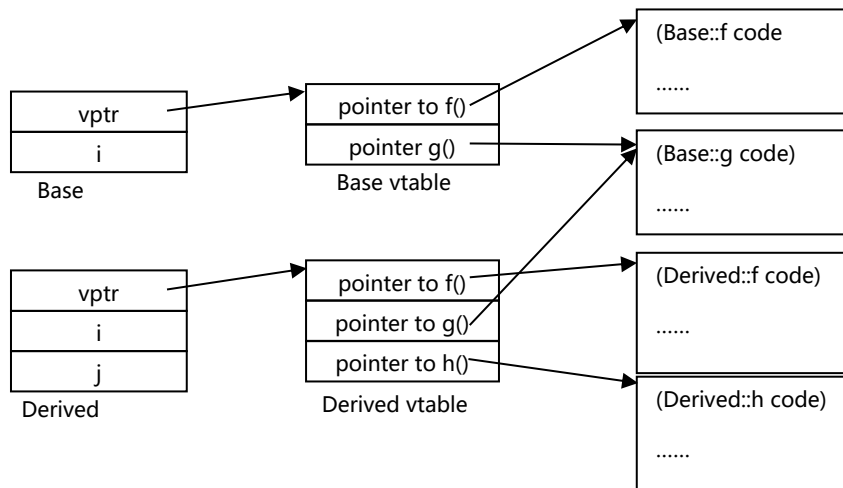
运行结果:

Base2::display()
Derived::display()

vtable

```
class Base {  
public:  
    virtual void f();  
    virtual void g();  
private:  
    int i;  
};
```

```
class Derived: public Base {  
public:  
    virtual void f(); //overwrite Base::f  
    virtual void h(); //新增的虚函数  
private:  
    int j;  
};
```



```
#include <iostream>
using namespace std;

class Base
{
public:
    int Bar(char x)
    {
        return (int)(x);
    }
    virtual int Bar(int x)
    {
        return (2 * x);
    }
};

class Derived : public Base
{
public:
    int Bar(char x)
    {
        return (int)(-x);
    }
    int Bar(int x)
    {
        return (x / 2);
    }
};

int main(void)
{
    Derived Obj;
    Base *pObj = &Obj;
    cout << pObj->Bar((char)(100));
    cout << pObj->Bar(100);
    return 0;
}
```

- ☐ A -100 50
- ☐ B 100 50
- ☐ C 100 200
- ☐ D -100 200

```
#include <iostream>
using namespace std;
class Base {
public:
    ~Base(); //不是虚函数
};
Base::~Base() {
    cout<< "Base destructor" << endl;
}
class Derived: public Base{
public:
    Derived();
    ~Derived(); //不是虚函数
private:
    int *p;
```



清华大学

```
Derived::Derived() {
    p = new int(0);
}
Derived::~~Derived() {
    cout << "Derived destructor" << endl;
    delete p;
}

void fun(Base* b) {
    delete b; //静态绑定, 只会调用~Base()
}

int main() {
    Base *b = new Derived();
    fun(b);
    return 0;
}
```

运行结果:
Base destructor

memory leakage

Virtual destructor

```
#include <iostream>
using namespace std;
class Base {
public:
    virtual ~Base();
};
Base::~~Base() {
    cout<< "Base destructor" << endl;
}
class Derived: public Base{
public:
    Derived();
    virtual ~Derived();
private:
    int *p;
};
```



清华大学

Derived destructor
Base destructor

```
Derived::Derived() {
    p = new int(0);
}
Derived::~~Derived() {
    cout << "Derived destructor" << endl;
    delete p;
}

void fun(Base* b) {
    delete b;
}

int main() {
    Base *b = new Derived();
    fun(b);
    return 0;
}
```